Activity 3 | Document Recommendation System

Short report - Path B: Embeddings

Objective

In this activity, we built a small recommendation system that helps locate related documents without having to open and review every file. We worked with a corpus of 25 sample documents: 13 about policies or processes and 12 about support incidents. The system is intended to return five relevant documents and a similarity score when someone starts from a document ID or describes a problem in their own words.

Description of the approach

We selected Path B (Embeddings) instead of relying only on matching keywords. Two documents may discuss the same issue but use different words, so we wanted the comparison to take their meaning into account. The Python script uses the pre-trained **sentence-transformers/all-MiniLM-L6-v2** model to create numeric representations of the documents.

The source information is stored in **corpus.csv**, with an ID, title, text and category for each entry. The script first checks that the required fields exist and that the IDs are not repeated. It then joins each document's title and text, generates its embedding, and compares vectors using cosine similarity. The results are ordered by score and the five highest are returned. When the search starts from an ID, that same document is excluded from the list.

For the execution interface, we included a simple console search where the user can paste text. The project also includes **example.py** to demonstrate a search by document ID. Both methods use the same embeddings and similarity calculation.

What we observed in the tests

We saved three console examples in the **evidencias** folder: a Policy/Process document, a Support/Incidents document and a free-text query. In the first two examples, a full document was pasted as the search text. The system placed that document at the top, which is expected because the input closely matches it. That behavior is different from searching by ID, where the code removes the starting document from the recommendations.

The free-text example was about account problems. The results pointed to account-related material and technical incidents, showing that the model can connect a short description with existing documents. Some results crossed categories, which may help when an incident and a policy deal with the same topic. Even so, a similarity score is a ranking measure, not a guarantee that the suggested document solves the problem.

Areas for improvement

What would need attention before using this in a company

Document information and coverage

The current corpus is small and contains simulated examples. With more documents, we would expect overlapping topics, outdated instructions and repeated cases. Adding metadata such as department, owner, update date and document status would make the results easier to filter. It would also help to keep older versions separate from active procedures and add new documents without rebuilding everything manually.

Relevance and evaluation

We tested whether the recommendations looked related to the input, but this is not yet a formal accuracy evaluation. A practical next step would be to ask users to mark which suggestions were actually helpful and measure how many relevant documents appear in the Top-5. We would also try shorter queries, synonyms and vague descriptions, since those can change the ranking. This would show where the model works well and where it needs adjustments.

Access control and maintenance

In a real organization, not every employee should receive every document. A production version would need permission checks before showing results, especially for internal policies, incidents or customer information. We would also cache saved embeddings, update them when documents change and monitor errors. These improvements would make the system safer and more practical as the collection grows.

Organizational value

This type of recommendation can reduce rework because employees can find previous decisions, procedures and incident solutions before starting an investigation from zero. For example, a technician handling a login issue could review related support cases and account policies instead of repeating the same checks. It can also support faster decisions by giving people a smaller set of relevant documents to review. The final decision still belongs to the person using the information, especially when a document may be incomplete or outdated.

Reflection video

Our short reflection on the organizational benefits, limitations and possible improvements is available here:

https://www.youtube.com/watch?v=jU7VCudr3XQ

Project repository: github.com/Guardian7366/documentVectorized